

Page 01

Race Condition pada Penjualan Stok – dari Bug Sepele ke Rusaknya Invariant

Bayangkan stok sparepart bengkel tinggal 1 unit. Dua kasir menekan "Bayar" bersamaan. Hasil akhir: 2 penjualan berhasil, stok 0. Invariant `initial == success + final` hancur.

Page 02

Lab 05 – Race Condition

Bagian ke-05 dari seri Senior Software Engineer Daily. Topik: race condition pada business state – bukan sekadar memory data race, melainkan interleaving READ → CHECK → WRITE di database atau shared mutable state.

Page 03

Kenapa Disebut Race Condition?

"Race" karena hasil bergantung pada siapa lebih cepat menulis. Timing di production (network, DB latency, concurrent user) memperbesar jendela masalah. Di dev, biasanya tidak muncul – karena tidak ada concurrent user.

Page 04

Mental Model Utama

Kode benar saat satu request ≠ benar saat banyak request berjalan bersamaan. Setiap `READ → CHECK → WRITE` pada shared mutable state adalah potensi race condition. Tanyakan: apa yang terjadi jika request lain mengubah data di antara langkah-langkah ini?

Page 05

Studi Kasus: Stok Oli Mesin

Satu unit Oli Mesin. Kasir A dan Kasir B baca stock = 1, keduanya cek > 0 → berhasil, kalkulasi new = 0, tulis stock = 0. Hasil: final_stock = 0, successful_sales = 2. Invariant: `1 != 2 + 0`.

Page 06

Lost Update – Formula Rusaknya State

`stale read` + `read-modify-write` + `concurrent execution` = `lost update`. Request B tidak tahu state sudah diubah oleh A. Perhitungan A (0) di-overwrite oleh B (0). Satu state transition efektif hilang.

Page 07

TOCTOU: Time-Of-Check To Time-Of-Use

Antara CHECK dan WRITE, state bisa berubah. Pola klasik: check slot kosong → beberapa milidetik berlalu → insert booking. Contoh kode: cek stock > 0, kurangi, simpan. Tanpa atomicity, dua transaksi bisa sama-sama lolos.

Page 08

Bukti di Unit Test: TestLostUpdate_Deterministic

Test di `lost\_update\_test.go` menggunakan channel barrier untuk memaksa interleaving

ving A READ → B READ → A WRITE → B WRITE. Result: `successful\_sales = 2`, `final\_stock = 0`, invariant rusak. Test PASS karena memang membuktikan kerentanan.

Page 09

Test In-Memory: AtomicUpdate Stock 1 Unit

500 goroutine berebut stok 1. Dengan `AtomicInventory` (mutex + check-then-decrement): success = 1, rejected = 499, final = 0. Invariant `initial == success + final` holds. Bukti perlindungan aplikasi-level bekerja.

Page 10

Test In-Memory: High Contention Stock 100

500 goroutine, stok awal 100. AtomicInventory menahan: success = 100, rejected = 400, final = 0. Invariant tetap kokoh. Deterministic barrier memastikan overlap ping tinggi – bukan flaky timing.

Page 11

Test Booking: 500 Concurrent untuk 1 Slot Eksklusif

Mock booking dengan UNIQUE key `branch:slot`. 500 goroutine → created = 1, conflict = 499, final = 1. Invariant `COUNT(bookings) == 1` terjaga. Ini bukti pattern check-then-insert tanpa constraint pasti gagal di production.

Page 12

Test Multi-Branch Booking

Branch-A dan Branch-B boleh booking slot waktu sama – karena invariant berlaku per kombinasi `(branch\_id, service\_date, slot\_time)`. Test `SameBranchDifferentBranch` membuktikan: same branch → conflict, different branch → allowed.

Page 13

Solusi 1 – Atomic Conditional Update

Satu statement SQL: `UPDATE inventory\_products SET stock = stock - 1 WHERE id = \$1 AND stock > 0 RETURNING stock`. 1 row affected = success, 0 rows = out of stock. Tanpa SELECT sebelum UPDATE, tanpa stale read window. Diimplementasikan di `atomic\_inventory.go`.

Page 14

Solusi 2 – Row Lock (Pessimistic)

`SELECT ... FOR UPDATE` mengambil row-level lock. Transaksi lain yang minta lock sama akan blocked sampai commit/rollback. Flow: A lock → B blocked → A commit → B unblock → baca stock = 0 → reject. Diimplementasikan di `postgres\_rowlock.go`.

Page 15

Bukti Row Lock via PostgreSQL Integration Test

`TestPostgresRowLock\_ConcurrentStock` menggunakan pg_stat_activity untuk membuktikan B benar-benar waiting (wait_event_type = 'Lock') sebelum A commit. Bukti bukannya inferansi – langsung diamati dari DB introspection.

Page 16

Solusi 3 – UNIQUE Constraint sebagai Last Defense

`tabel service\_bookings` punya `UNIQUE(branch\_id, service\_date, slot\_time)`. Ketika dua INSERT bersamaan, salah satu mendapat SQLSTATE 23505 (unique_violation). Constraint absolut – engine database yang menolak duplikat.

Page 17

Bukti UNIQUE Constraint di DB Test

`TestPostgres\_ConcurrentBooking`: 500 goroutine, created = 1, conflict (23505) = 499. Berbeda dari application check, constraint tidak bisa dijejol race condition. Ini final safety net – bahkan jika ada bug di aplikasi.

Page 18

Optimistic Locking – Version Check

UPDATE dengan kondisi version: `SET stock = \$1, version = version + 1 WHERE id = \$2 AND version = \$old`. 0 rows affected berarti state sudah berubah, aplikasi bisa retry. Cocok untuk contention rendah. Lab ini tidak implementasi penuh – lihat lab 93-optimistic-locking.

Page 19

Distributed Lock – Bukan Solusi Default

Gunakan hanya jika koordinasi melintas proses/instance/node/eksternal API. Jika invariant bisa dijaga database (atomic, row lock, unique), jangan pakai Redis lock. Kompleksitas tambahan: ownership, TTL, safe unlock, split-brain, network partition.

Page 20

Tabel Keputusan: Distributed Lock vs Database Constraint

Use Case	Rekomendasi
Stock / Booking / Invoice Number (single DB)	Atomic Update / Row Lock / UNIQUE
Koordinasi multi-replica / eksternal API	Distributed Lock
Mencegah double-click pembayaran	Idempotency key

Page 21

Transaction Saja Tidak Cukup

`BEGIN ... COMMIT` tidak otomatis hilangkan race condition pada READ COMMITTED. Dua transaksi bisa baca stock = 1 bersamaan, keduanya update ke 0. Correctness bergantung pada query pattern, lock, dan isolation semantics.

Page 22

go test -race ≠ Bebas Race Condition Bisnis

Race detector Go hanya mendeteksi memory-level concurrent access tanpa sinkronisasi. Business race condition pada database (lost update) lolos dari detector – karena variabel di memory sudah thread-safe. Test "Unsafe" justru sengaja PASS saat invariant hancur.

Page 23

Kesalahan Umum yang Perlu Dihindari

1. Hanya validasi frontend – bypass mudah via API.
2. Tidak pakai Unique Constraint – andalkan SELECT sebelum INSERT.
3. Anggap BEGIN/COMMIT cukup – lupa row lock atau isolation.
4. Mutex untuk microservices – hanya mengunci satu instance, bukan semua server.

Page 24

Software Engineering Lab

Repository: <https://github.com/lukman-ss/software-engineering-lab>

Lab: labs/05-race-condition

URL: <https://github.com/lukman-ss/software-engineering-lab/tree/main/labs/05-race-condition>

#tagar: #RaceCondition #Concurrency #Database #SoftwareEngineering #Go